cloud campus

situation professionnelle de mise en production et de maintenance applicative

La mise en situation professionnelle de maintenance d'une application



Présentation de l’environnement de l’application

L’environnement fourni comprend une application web existante. Elle se compose principalement :

✓ D’un front-end web “ReactJS”

✓ D’un back-end “NodeJS”

✓ D’une base de données NoSQL “MongoDb”

✓ D’un micro-service “NodeJS”

✓ D’appels API externes

Cette application comporte des bugs techniques, des failles de sécurité et repose sur certains composants obsolètes.

Rappel Session 1 (durée de trois heures)

solution d'hébergement & maintenance

La session 1 est subdivisée en plusieurs étapes dont l'objectif est la préproduction, la mise en production et la supervision de l'application

✓ E21 : Le choix d'une solution d'hébergement de type cloud adaptée à l'application

✓ E22 : La mise en oeuvre sécurisée de l'environnement de production

✓ E23 : La réservation d'un nom de domaine, la configuration des DNS correspondants, l'installation et la vérification des certificats de sécurité pour les environnements de pré-production et de production.

✓ E24 - La mise en oeuvre d'un système de déploiement automatisé (CI/CD - conteneurisation)

✓ E25 - La mise en oeuvre de la journalisation et d'outils d'audit correspondants aux deux environnements

✓ E26 - La mise en oeuvre des outils de supervision et la configuration des alertes adaptées au contexte de l'application pour les deux environnements précédemment créés.

Rappel Session 2 (2 heures)

code robuste, sécurité et documentation

- **E27** – La détection des bugs de l'application et la proposition de mesures correctives.
- **E28** - La détection des failles de sécurité, et la proposition de mesures correctives.
- **E29** - La génération de la documentation à partir du code source et la mise à jour du journal des évolutions ou « changelog ».

L’application web : Présentation



L’application *: Le projet web à mettre en production et à maintenir est tout simple; il s’agit d’un mini site e-commerce avec les fonctionnalités de :*

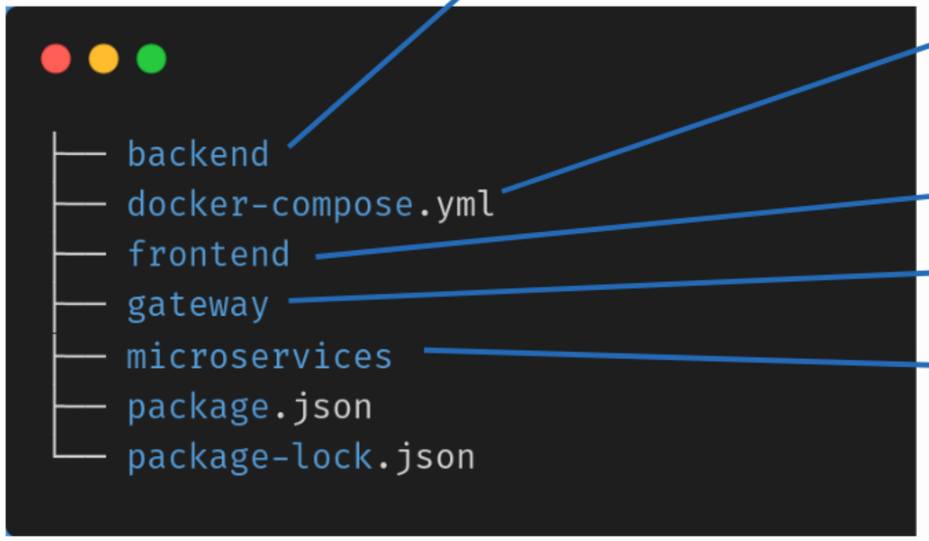
- *Liste des produits*
- *bouton d’ajout au panier*
- *contexte de gestion du panier global avec context API*
- *fonctionnalités de gestion d’un panier (CRUD produits dans le panier, bouton validation de la commande) avec context API*
- *l’ecran synthèse commande avant la validation d’une commande, gestion de l’état d’une commande “En cours de traitement”, “Expédié” …*
- *Gestion des utilisateurs avec les rôles “user”, “admin”*
  - *user : client du site (navigation, commande …)*
  - *admin : gestion du backend (commande, produit, stock, utilisateurs …)*
- *Microservices : pour la gestion des notifications (événement mise à jour stock et validation commande …)*
- *Gateway : qui lie l’ensemble de l’application (front & back) aux microservices*

L’application à été développé en utilisation le stack ***MERN***

L’application web : structure

La structure de repertoire et fichier de ce projet :





Contenant l’API de l’application en liaison avec la base de données

conteneurisation : début de construction de l’orchestration avec docker compose

Le front de l’application en React

Le gateway met en relation le fonctionnement de l’application et des microservices

Microservices, fonctionnalités specifiques de l’application gerées sous forme de microservices (notifications, stock-management ...)

L’application web : structure - backend

La structure de repertoire et fichier de ce projet :




```
backend/
├── config
├── controllers
├── middlewares
├── models
├── node_modules
├── package.json
├── package-lock.json
├── routes
├── seeder.js
└── server.js
```

contient la configuration et la connexion à la base de données **(db.js)**

contient l’ensemble des controllers de l’application : admin, auth, order, product ... controller pour le traitement des demandes CRUD

Un seul middleware (authMiddleware.js) contenant deux fonctions : authenticateToken et isAdmin

L’ensemble des modèles données nécessaires dans l’application => Order, Product, User ...

L’ensemble des routes de l’application => admin, auth, order et product Route

Script de création d’un jeux de données

L’application web : structure - frontend

La structure de repertoire et fichier de ce projet :




```
frontend/
├── package.json
├── package-lock.json
├── postcss.config.js
├── public
│   ├── favicon.ico
│   ├── index.html
│   ├── logo192.png
│   ├── logo512.png
│   ├── manifest.json
│   └── robots.txt
├── README.md
├── src
│   ├── App.css
│   ├── App.js
│   ├── App.test.js
│   ├── components
│   ├── context
│   ├── index.css
│   ├── index.js
│   ├── logo.svg
│   ├── pages
│   ├── reportWebVitals.js
│   ├── services
│   └── setupTests.js
└── tailwind.config.js
```

L’application web : structure - gateway

La structure de repertoire et fichier de ce projet :




```
gateway/
├── package.json
├── package-lock.json
├── routes
│   ├── auth.js
│   ├── notifi.js
│   ├── notif.js
│   └── stock.js
└── server.js
```

L’application web : structure - microservices

La structure de repertoire et fichier de ce projet :




```
microservices
├── auth-gateway
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   └── routes
├── notifications
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   └── routes
└── stock-management
    ├── index.js
    ├── package.json
    ├── package-lock.json
    └── routes
```

L’application web : Les écrans - Accueil

Page “accueil” de l’application





L’application web : Les écrans - Panier

Page “panier” de l’application



Mon Application

Panier 2 Connexion Inscription

Mon Panier

Produit	Prix Unitaire (€)	Quantité	Prix Total (€)	Actions
Ordinateur Portable	1000	- 1 +	1000.00	Retirer
Smartphone	700	- 1 +	700.00	Retirer

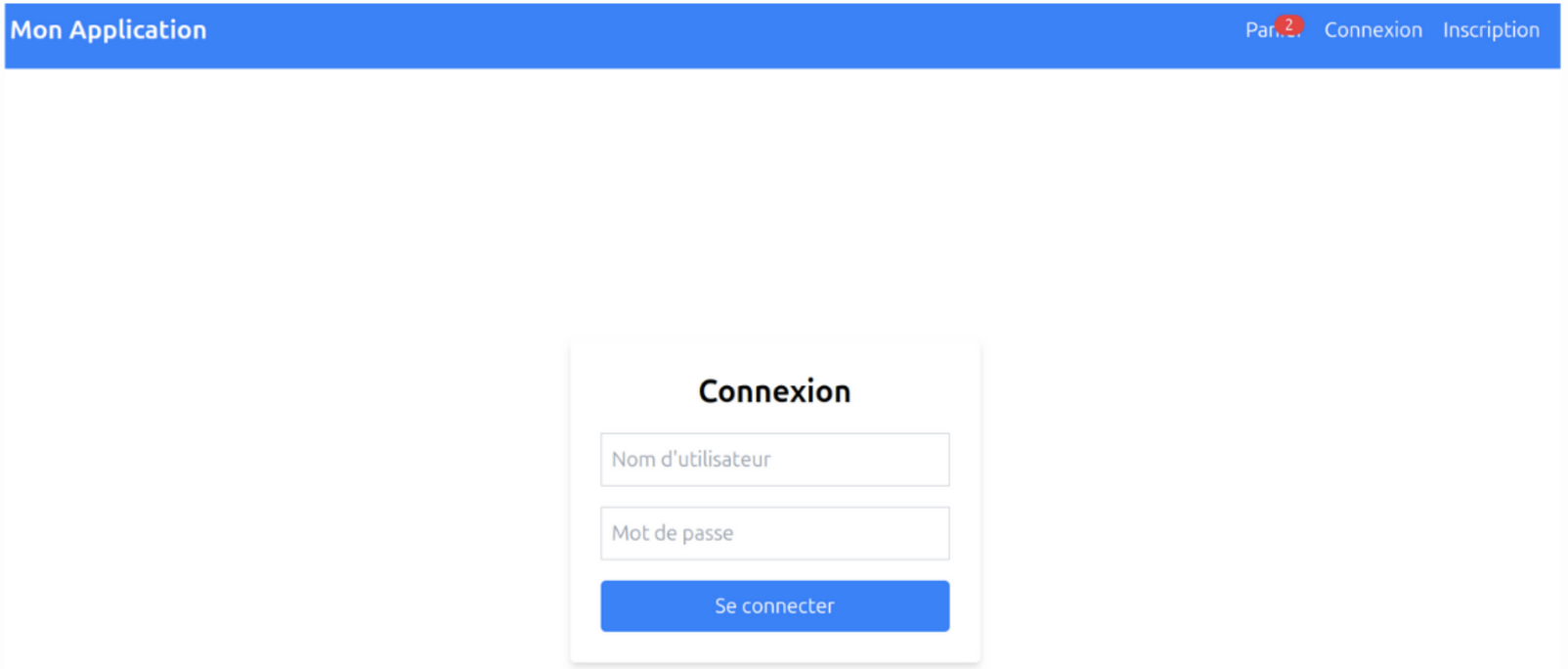
Total : 1700.00 €

Vider le panier | Passer une commande

L’application web : Les écrans - Connexion

Page “connexion” de l’application

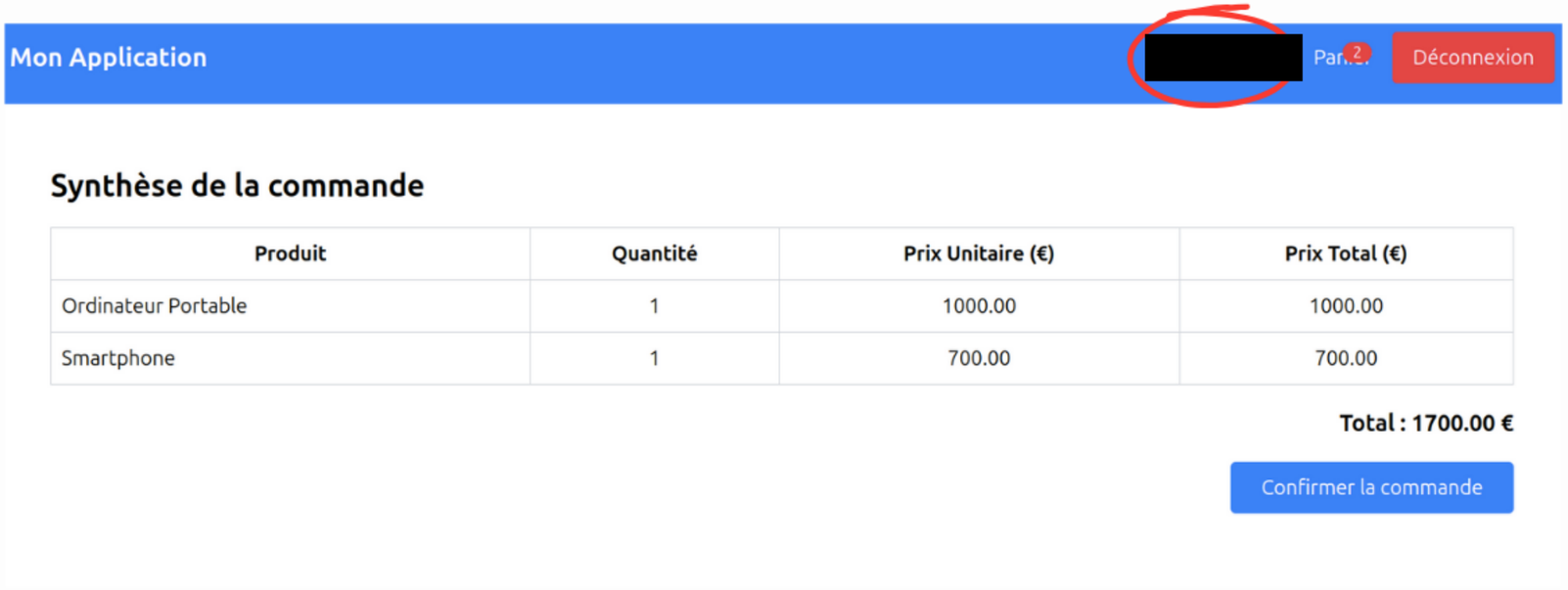




L’application web : Les écrans - Commande

Page “après connexion” de l’application





L’application web : Les écrans - Adresse, Paiement, transport

Page “Adresse de livraison et Méthode de paiement” de l’application

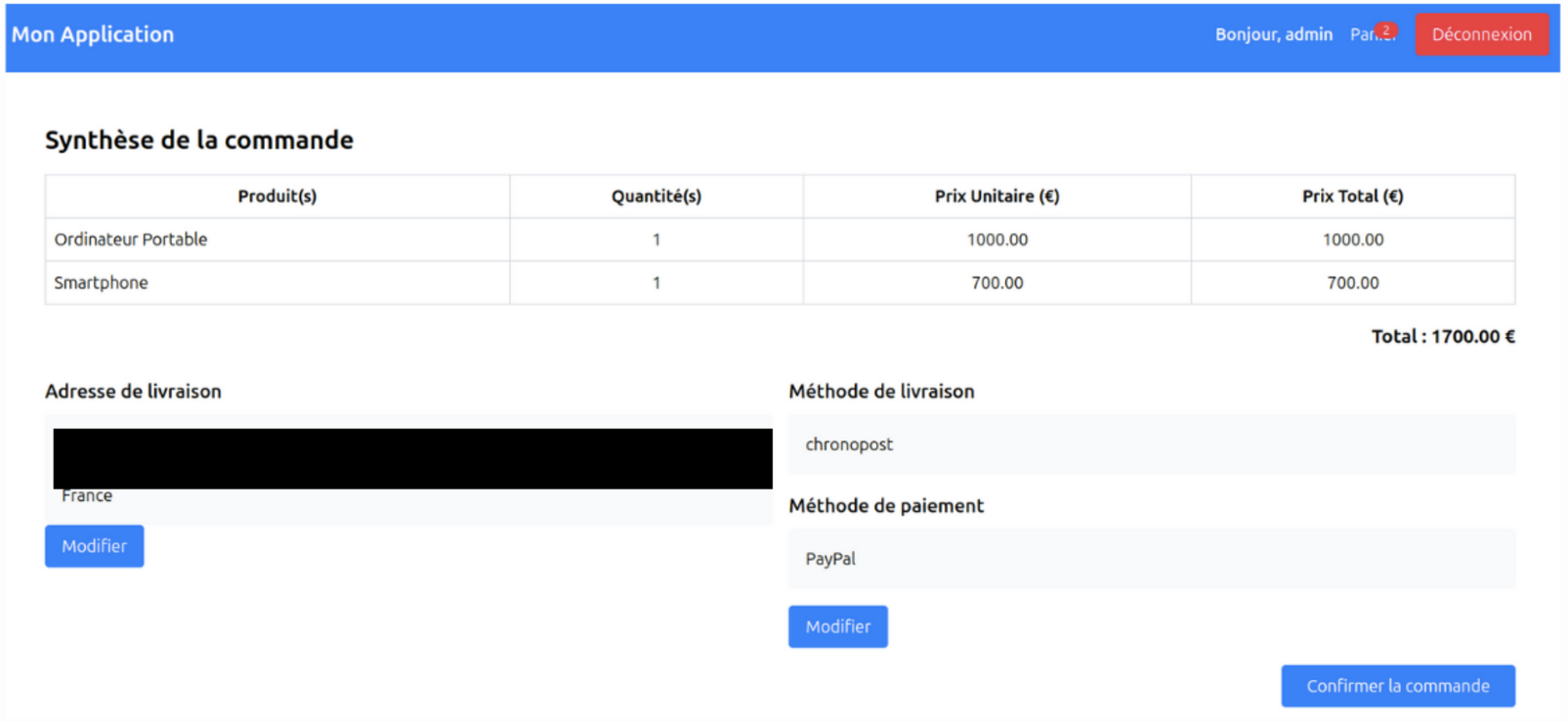




L’application web : Les écrans - Adresse, Paiement, transport

La structure de répertoire et fichier de ce projet :

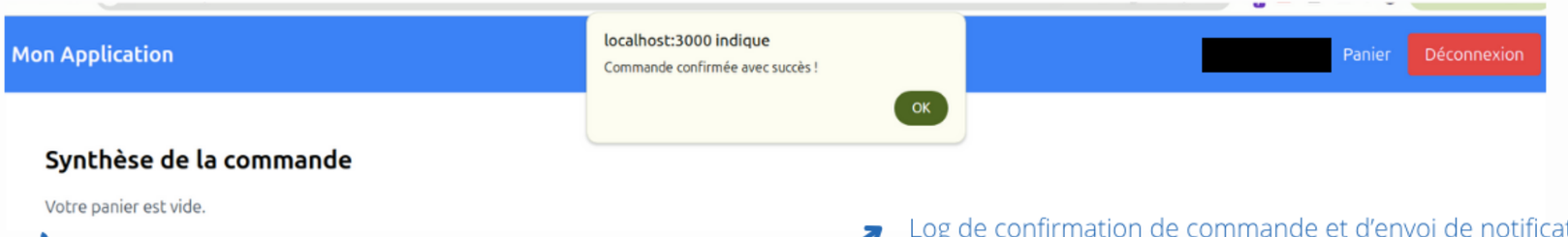




L’application web : Les écrans - Adresse, Paiement, transport

Action de confirmation de commande

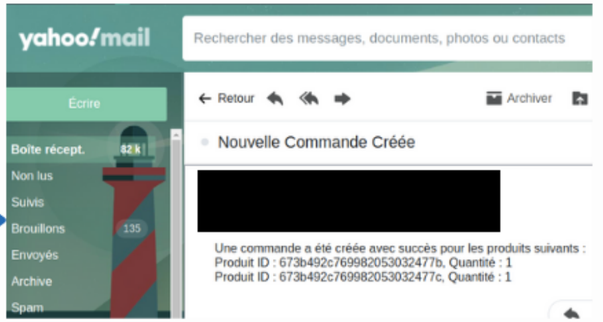




Log de confirmation de commande et d’envoi de notification mail

```
Commande sauvegardée : {
  userId: new ObjectId('673b645b2fd6b4af9bc49e4e'),
  items: [
    {
      productId: new ObjectId('673b492c769982053032477b'),
      quantity: 1,
      price: 1000,
      _id: new ObjectId('67c82c884de76a1aaad57291')
    },
    {
      productId: new ObjectId('673b492c769982053032477c'),
      quantity: 1,
      price: 700,
      _id: new ObjectId('67c82c884de76a1aaad57292')
    }
  ],
  total: 1700,
  status: 'En attente',
  shippingAddress: {
    street: [redacted]
    city: [redacted]
    postalCode: [redacted]
    country: 'France'
  },
  paymentMethod: 'PayPal',
  shippingMethod: 'chronopost',
  _id: new ObjectId('67c82c884de76a1aaad57290'),
  createdAt: 2025-03-05T10:50:48.736Z,
  updatedAt: 2025-03-05T10:50:48.736Z,
  __v: 0
}
to is [redacted] subject is Nouvelle Commande Créée text is Une commande a été créée avec succès pour les produits suivants :
Produit ID : 673b492c769982053032477b, Quantité : 1
Produit ID : 673b492c769982053032477c, Quantité : 1
Email envoyé avec succès
```

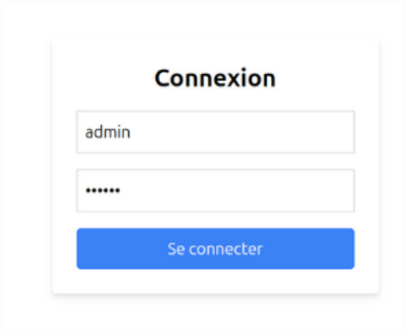
Notification mail du microservice => nouvelle commande

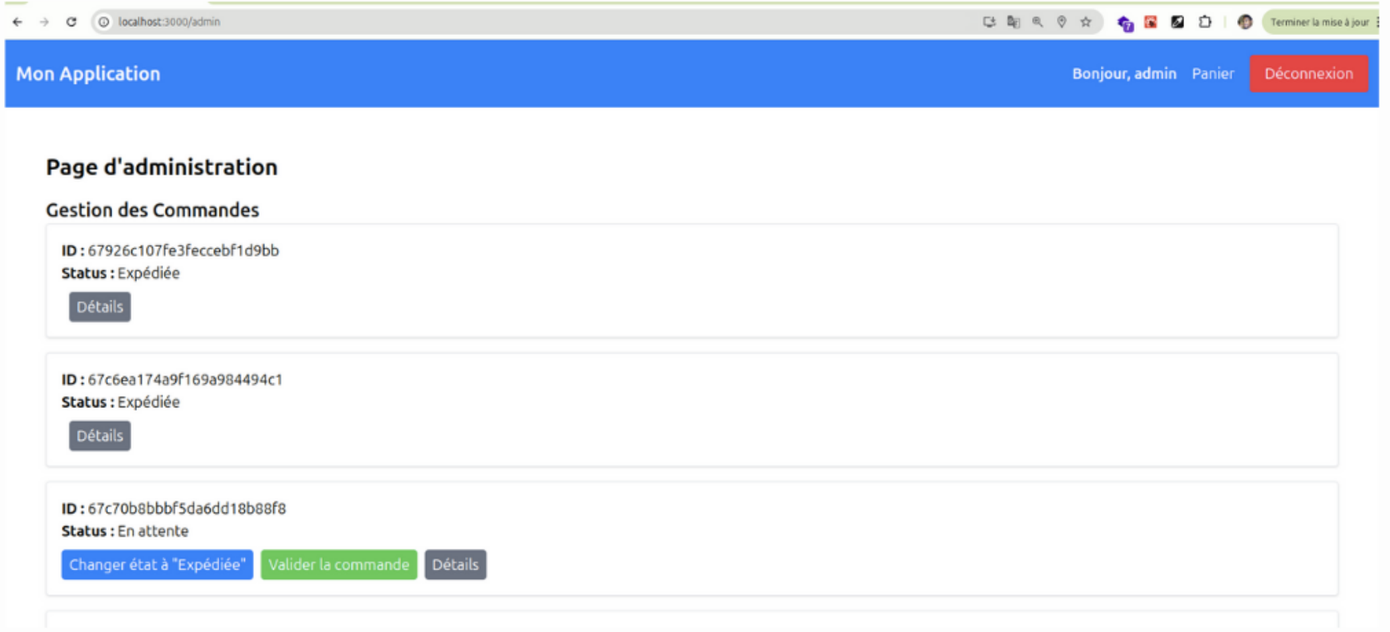


L’application web : Les écrans - Adresse, Paiement, transport

Pages “connexion admin et administration”

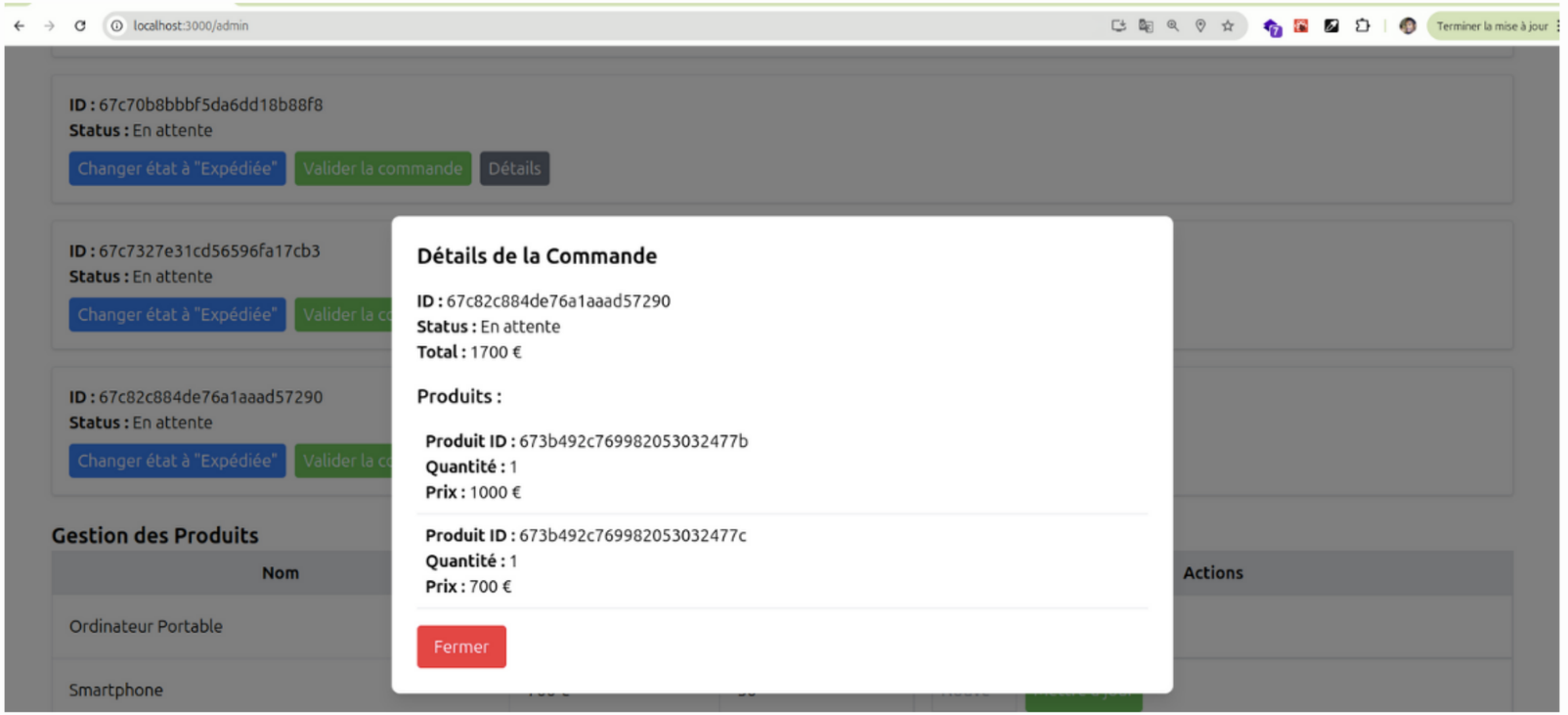






L’application web : Les écrans - Adresse, Paiement, transport

Action détails de la commande en admin



L’application web : Les écrans - Adresse, Paiement, transport

Section gestion des produits en admin



Gestion des Produits

Nom	Prix	Stock	Actions
Ordinateur Portable	1000 €	22	Nouve Mettre à jour
Smartphone	700 €	50	Nouve Mettre à jour
Casque Audio	150 €	30	Nouve Mettre à jour
Tablette	400 €	25	Nouve Mettre à jour
Clavier Mécanique	120 €	15	Nouve Mettre à jour

MicroService : Notification et Stock management



Notification :

Ce service utilise la bibliothèque NodeMailer pour envoyer des notifications mail aux administrateurs et utilisateurs du site e-commerce



Stock management

Ce service s'occupe de l'aspect de gestion du stock de l'application e-commerce : l'objectif est de decentraliser toute la gestion de stock dans ce service

Gateway : point d’entré de microservices



Gateway :

```
app.use('/notify', notifiProxy);
app.use('/update-stock', stockProxy);
```

Le gateway personnalise le chemin d’accès aux microservices , nous pouvons ainsi utiliser le même domaine pour accéder à l’ensemble des microservices :

- http://localhost:8000/notify => pour accéder au service de notification
- http://localhost:8000/update-stock => pour accéder au service de gestion du stock

Execution de l'application en Local

Actuellement, pour faire fonctionner l'application en local, il faut procéder de deux manières :

- Exécuter individuellement chaque composant : Backend, Microservices, Le gatway, et enfin le front
- Ou sinon utiliser le script ci dessous pour lancer l'ensemble des composants en une seule commande

1

Le Backend : node server.js

```
> node server.js
Serveur en écoute sur le port 5000
MongoDB connecté
```

le gateway et les microservices : npm run start:all

```
{
    "scripts": {
      "start:notifications": "node microservices/notifications/index.js",
      "start:stock-management": "node microservices/stock-management/index.js",
      "start:gateway": "node gateway/server.js",
      "start:all": "npm-run-all --parallel start:notifications start:stock-management start:gateway"
    }
  }
```

Le Frontend : npm start

```
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  }
```

2

package.json > ...

```
{
    Debug
  "scripts": {
    "start:notifications": "nodemon microservices/notifications/index.js",
    "start:stock-management": "nodemon microservices/stock-management/index.js",
    "start:gateway": "nodemon gateway/server.js",
    "start:backend": "nodemon backend/server.js",
    "start:frontend": "npm --prefix frontend start",
    "start:all": "npm-run-all --parallel start:notifications start:stock-management start:gateway start:backend start:frontend"
  }
}
```



```
npm run start:all
```

Évaluation 1 & 2 : objectif

L'objectif de cette évaluation est de vérifier et valider vos compétences en matière de gestion, de mise en production et de maintenance d'une application web quelconque. Vous disposez à présent d'une application non robuste (bug, sécurité, contrôle d'accès ...) mais basique en terme de fonctionnalité et de structure

- ***fonctionnalité basique*** *: site e-commerce (gestion utilisateur avec rôle, gestion de panier, gestion de commande avec choix d'un mode paiement et de transporteur)*
- ***structure basique*** *: frontend, backend, gateway avec microservices*

Dans le cadre de votre évaluation (voir document en pièce jointe et slide 3 et 4 : suivre les étapes E21 jusqu'a E29) , vous devez :

- *Trouver et Corriger les* ***bugs de sécurité****, faire de proposition amélioration et les réaliser*
- *Rendre le code* ***robuste*** *en effectuant un* ***contrôle des données*** *utilisateurs*
- *Vérifier et* ***mettre à jour les bibliothèques*** *dans l'ensemble du projet*
- *Créer un environnement de* ***préproduction*** *(conteneurisation - docker) et de* ***production***
- *Faire le* ***choix d'un hébergeur*** *en fonction de critères pertinents de performance, de maintenance, de sécurité ...*
  - *Utilisation de* ***configuration de nom de domaine et de sous domaine*** *pour les composants de l'application (configuration DNS, mise en place et vérification de certificats de sécurité pour les deux environnements) afin que l'ensemble front, back, microservice ... fonctionnent*
- *Mise en place d'un* ***CI / CD*** *pour les deux environnements pour* ***automatiser*** *la mise en production*
- *Mise en place d'outils de journalisation et d'audits pour les deux environnements*
- *Mise en oeuvre et utilisation d'outils de supervision de service et configuration de sondes d'alertes dans le contexte de l'application pour les deux environnements*
- *Documentation du code de l'application : génération d'une* ***documentation*** *à partir du code source et de la* ***journalisation*** *des évolutions*

NB : illustrations et consignes sur les slides de 27 à 35

Mise à disposition et Rendu final

Vous aurez à disposition pour cette évaluation :

- *Le code source de l'application : backend, frontend, gateway, microservices, dump base de données*
- *Les accès admin du premier utilisateur de la base de données*
- *Les détails SMTP pour l'envoi de mail - notification*

Pour l'ensemble des tâches réalisées, veuillez fournir un document contenant :

- *Les différentes tâches effectuées sous forme de séquences détaillées*
- *Des explications approfondies*
- *Des captures d'écran illustratives*

Enfin, merci de transmettre tous les accès et identifiants (serveurs, comptes d'accès, etc.) afin de permettre la vérification des travaux effectués.

Sécurité et robustesse du code

Utilisation des variables d'environnement (.env)

- *Éviter d'exposer les informations sensibles (clé API, URI MongoDB...).*
- *Stocker ces données dans un fichier .env et les charger avec dotenv.*

.env



.gitignore



Sécurité et robustesse du code

Contrôle des données et validation des entrées utilisateur

- *Éviter d'insérer des données non conformes dans la base de données.*
- *Utiliser Joi pour définir des schémas de validation.*



```
const Joi = require('joi');

const registerSchema = Joi.object({
  username: Joi.string().min(3).max(30).required(),
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required()
});

// Exemple de validation d'une requête utilisateur
app.post('/register', (req, res) => {
  const { error } = registerSchema.validate(req.body);
  if (error) return res.status(400).json({ error: error.details[0].message });

  res.send('Inscription réussie !');
});
```

Sécurité et robustesse du code

Éviter localStorage pour stocker des tokens sensibles

- ***Mauvaise pratique*** *: Stocker un token JWT dans localStorage, car il peut être exposé aux attaques XSS*
- ***Bonne pratique*** *: Utiliser un cookie HTTPOnly sécurisé.*



```
const session = require('express-session');
const cookieParser = require('cookie-parser');

app.use(cookieParser());
app.use(session({
  secret: process.env.JWT_SECRET,
  resave: false,
  saveUninitialized: true,
  cookie: { secure: true, httpOnly: true }
}));
```

Sécurité et robustesse du code

Gestion des erreurs avec try/catch

- *Éviter le crash du serveur à cause d'une erreur non gérée*
- *Entourer les opérations critiques avec try/catch et vérifier en front les retours serveur*



```
app.get('/user/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'Utilisateur non trouvé' });
    res.json(user);
  } catch (error) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
});
```

Sécurité et robustesse du code

Afficher des messages côté client

- *Utilisez des modules comme connect-flash pour envoyer des messages temporaires aux utilisateurs après un événement du backend vers le frontend*



```
app.get('/user/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'Utilisateur non trouvé' });
    res.json(user);
  } catch (error) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
});
```

Sécurité et robustesse du code

Qualité du code avec ESLint et Prettier

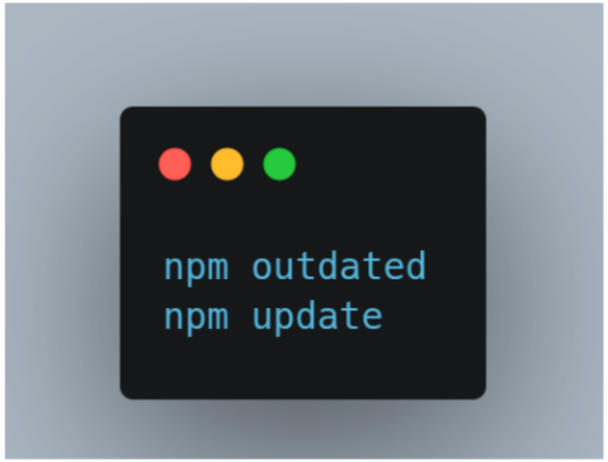
- *Installez et configurez les outils d'assistance et d'amélioration de la qualité du code ESLint et Prettier*



Sécurité et robustesse du code

Maintien des bibliothèques à jour et journalisation

- *Vérifiez et mettez à jour les dépendances*
- *Utilisez des packages comme Winston pour la gestion des logs*




```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [
    new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
    new winston.transports.File({ filename: 'logs/combined.log' })
  ]
});

logger.info('Application démarrée');
```

Sécurité et robustesse du code

Utilisation de Debug pour l'assistance Dev

```
npm install debug
```

```
const debug = require('debug')('app:debug');

debug('Mode développement activé');
```

```
DEBUG=app:debug node server.js
```

ATTENTION CONSIGNE

« L'UTILISATION DE L'IA ET D'INTERNET EST AUTORISÉE, MAIS L'USAGE D'OUTILS DE COMMUNICATION TELS QUE WHATSAPP OU DISCORD EST INTERDIT. »



Cloud Campus

BON COURAGE :-)

